INT1405 - DISTRIBUTED SYSTEMS

LECTURE 07

Consistency Models in Distributed Systems

Semester 1, 2026-2027

Lê Hà Thanh
lehathanh@ptithcm.edu.vn

Tanenbaum & van Steen (2nd Edition),
Chapter 7 (Sections 7.1–7.3)

Slide 02: Learning Objectives & Operational Roadmap

Core Pedagogical Objectives

• Analyze Data-Centric Models: Differentiate Strict, Sequential, Causal, FIFO, Weak, Release, and Entry consistency.

• Evaluate Client-Centric Models: Master Monotonic Reads, Monotonic Writes, Read-Your-Writes, and Writes-Follow-Reads.

• Trace Execution Anomalies: Diagnose stale reads, out-of-order writes, and data divergence in distributed execution traces.

• Master Vector Timestamps: Apply session vectors (V_read, V_write) to enforce client-centric consistency guarantees.

Lecture Roadmap

• Module 1: Foundations of Consistency & Continuous Consistency (Slides 01-08).

• Module 2: Strict, Sequential & Causal Consistency Models (Slides 09-16).

• Module 3: FIFO, Weak, Release & Entry Consistency Models (Slides 17-24).

• Module 4: Client-Centric Consistency Models & Session Vectors (Slides 25-32).

• Module 5: Real-World Systems, Review Problems & Practical Lab (Slides 33-40).

Content

- Module 1: Foundations of Consistency & Continuous Consistency (Slides 01-08).
- Module 2: Strict, Sequential & Causal Consistency Models (Slides 09-16).
- Module 3: FIFO, Weak, Release & Entry Consistency Models (Slides 17-24).
- Module 4: Client-Centric Consistency Models & Session Vectors (Slides 25-32).
- Module 5: Real-World Systems, Review Problems & Practical Lab (Slides 33-40).

Slide 03: The Fundamental Trade-off: Replication vs. Consistency

Motivations for Data Replication

- **Reliability & Fault Tolerance**: Maintaining multiple data copies prevents data loss when hardware nodes crash.
- **Performance Scaling**: Placing data replicas close to geographically dispersed users reduces round-trip network latency.
- **Read Load Balancing**: Distributes read queries across N replicas to prevent CPU and network interface bottlenecks.

The Price of Replication

- **Consistency Challenges**: Modifying one copy creates immediate divergence across replicas.
- **Network Overhead**: Pushing updates synchronously across WAN links degrades write performance and throughput.
- **Core Design Conflict**: High availability and ultra-low read latency directly conflict with strict global data consistency.

Slide 04: CAP Theorem & PACELC Theoretical Frameworks

The CAP Theorem (Brewer, 2000)

- **Consistency** (C): Every read returns the most recent write or an error.
- **Availability** (A): Every non-failing node returns a non-error response without guarantee of latest write.
- **Partition Tolerance** (P): System operates despite dropped or delayed network messages.
- **Theorem Limit**: Under network partition P, a distributed store MUST choose between Consistency C or Availability A.

The PACELC Theorem Extension (Abadi, 2012)

- If Partition (P): Trade off Availability (A) or Consistency (C).
- Else (E) Normal Execution: Trade off Latency (L) or Consistency (C).
- **PC/EC Systems**: Prioritize consistency during partitions and normal run-time (e.g., Google Spanner).
- **PA/EL Systems**: Prioritize availability and low latency over strict consistency (e.g., Apache Cassandra, Dynamo).

Slide 05: Data-Centric Consistency Abstraction & Contract Model

The Data Store Contract Abstraction

- **System Contract**: A consistency model is a formal contract between executing process applications and a distributed data store.
- **Contract Guarantee**: If processes agree to follow specific rules for accessing data, the store guarantees predictable read results.
- **Data Store Model**: Shared data store physically distributed across multiple processes, each holding a local copy.

Standard Operation Notation

- $W_i(x)a$: Process P_i writes value 'a' to data item x.
- $R_i(x)b$: Process P_i reads data item x, returning value 'b'.
- **Time Vector**: Execution timeline progresses horizontally from left to right.
- **Initial State**: All data items initialized to NIL unless stated otherwise.

Slide 06: Continuous Consistency Ranges (Yu & Vahdat)

The Spectrum of Consistency

- **Binary Fallacy**: Consistency is not merely a binary choice between strict lockstep consistency and total eventual chaos.
- **Continuous Consistency Model**: Allows application developers to define custom acceptable bounds on inconsistency.
- **Conit Abstraction**: Consistency Unit specifying the fine-grained or coarse-grained data subset over which consistency is measured.

3 Independent Inconsistency Axes

- 1. **Numerical Deviation**: Bounds absolute or percentage difference between replica values (e.g., stock price differs by < $0.02).
- 2. **Staleness Deviation**: Bounds the physical or logical time elapsed since a replica was last updated (e.g., data < 10s old).
- 3. **Ordering Deviation**: Bounds the maximum length of tentative, uncommitted update queues at any replica (e.g., <= 3 uncommitted writes).

Slide 07: The Notion of a Conit (Consistency Unit)

Defining Conits in Practice

- **Conit Definition**: A logical grouping of data items that defines the granularity of consistency enforcement.
- **Example Conit**: A single stock record, a user shopping cart, or a weather report item.
- **Vector Clock Tracking**: Each replica maintains a vector clock tracking committed vs. tentative updates per conit.

Conit Granularity Trade-offs

- **Coarse-Grained Conits** (e.g., entire database): High update aggregation, but frequent false sharing and premature threshold trips.
- **Fine-Grained Conits** (e.g., individual attributes): Low false sharing, but high vector tracking and memory overhead.
- **Developer API**: Exposed via library calls (e.g., AffectsConit(ConitQ, 1, 1)).

Slide 08: Conit Granularity & False Sharing Overhead

False Sharing in Coarse Conits

- **Mechanism**: Occurs when independent, unrelated data items are grouped into a single coarse conit.
- **Problem**: Updating Data Item A trips the conit's inconsistency threshold, forcing unnecessary update flushes for unrelated Data Item B.
- **Performance Impact**: Increases network sync message frequency even when data items are logically independent.

Fine-Grained Conit Trade-off Analysis

- **Isolation Benefit**: Eliminates false sharing; replicas sync only when specific related items change.
- **Management Cost**: Exponential increase in the number of active conit vector clocks maintained by replica servers.
- **Design Goal**: Balance conit size to match application data access boundaries.

Content

- Module 1: Foundations of Consistency & Continuous Consistency (Slides 01-08).
- Module 2: Strict, Sequential & Causal Consistency Models (Slides 09-16).
- Module 3: FIFO, Weak, Release & Entry Consistency Models (Slides 17-24).
- Module 4: Client-Centric Consistency Models & Session Vectors (Slides 25-32).
- Module 5: Real-World Systems, Review Problems & Practical Lab (Slides 33-40).

Slide 09: Strict Consistency: Absolute Physical Time Ordering

Formal Definition of Strict Consistency

• **Physical Time Bound**: Any read on data item x returns a value corresponding to the result of the most recent write on x in physical time.

• **Mathematical Requirement**: For all $R_i(x)v$, v must equal the value written by the latest $W_j(x)v$ in physical time t.

• **Global Synchronization**: Assumes an absolute, perfectly synchronized global clock across all processes.

Practical Impossibility in Real Networks

• **Speed-of-Light Limit**: Instantaneous update propagation across physical distances is physically impossible.

• **Asynchronous Delay**: Network packet jitter and clock drift prevent absolute global physical ordering.

• **Role**: Serves as a theoretical ideal rather than a practical protocol implementation.

Slide 10: Strict Consistency Physical Impossibility in Real Networks

Why Strict Consistency Fails in Practice

• Scenario: Process P1 writes $W(x)a$ at physical time t_1 on Node 1. Process P2 reads $R(x)$ at time $t_1 + 1$ns on Node 2 across WAN.

• Physical Constraint: Light travels ~30cm per nanosecond. Signal cannot cross datacenters in 1ns.

• Outcome: Node 2 reads stale value NIL or old state, violating strict physical consistency.

Transition to Logical Consistency Models

• Physical Time Abandonment: Distributed systems drop physical time requirements in favor of logical event orderings.

• Logical Ordering Focus: Focuses on agreed operation orderings rather than wall-clock timestamps.

• Foundational Model: Leads directly to Leslie Lamport's Sequential Consistency.

Slide 11: Sequential Consistency (Lamport, 1979)

Formal Definition of Sequential Consistency

• Lamport's Rule (1979): The result of any execution is the same as if the operations of all processes were executed in some sequential order.

• Program Order Constraint: The operations of each individual process appear in this sequence in the order specified by its program.

• Interleaving Agreement: ALL processes must observe the exact same global interleaved sequence of operations.

Key Properties of Sequential Consistency

• No Physical Time Requirement: Operations do not need to be ordered by wall-clock time.

• Global Consensus: If Process P_3 sees write 'a' before write 'b', then Process P_4 MUST also see 'a' before 'b'.

• Programmability: Provides intuitive semantics for concurrent application developers.

Slide 12: Sequential Consistency Valid vs. Invalid Trace Analysis

Trace Example 1: Valid Sequential Execution

- P_1: $W(x)a$
- P_2: $W(x)b$
- P_3: $R(x)b \rightarrow R(x)a$
- P_4: $R(x)b \rightarrow R(x)a$

- **Analysis**: VALID!
Both P_3 and P_4 observe write 'b' before write 'a' (interleaving sequence: $W(x)b \rightarrow W(x)a$).

Trace Example 2: Invalid Sequential Execution

- P_1: $W(x)a$
- P_2: $W(x)b$
- P_3: $R(x)b \rightarrow R(x)a$
- P_4: $R(x)a \rightarrow R(x)b$

- **Analysis**: INVALID!
P_3 sees 'b' before 'a', while P_4 sees 'a' before 'b'. Disagreement on global order violates sequential consistency.

Slide 13: Causal Consistency: Preserving Cause and Effect

Formal Definition of Causal Consistency

- **Causal Requirement**: Writes that are potentially causally related must be seen by every process in the same order.
- **Concurrent Operations**: Writes that are NOT causally related (concurrent) may be seen in different orders by different processes.
- **Weakening Sequential**: Relaxes global ordering constraints on concurrent operations to improve performance.

Establishing Causal Dependencies

- **Local Program Order**: $W_1(x)a$ before $W_1(x)b$ on same process P_1 implies $W_1(x)a \rightarrow W_1(x)b$.
- **Read-to-Write Chain**: If P_2 reads $R_2(x)a$ and then issues $W_2(y)b$, then $W_1(x)a$ causally precedes $W_2(y)b$.
- **Transitivity**: If $Op1 \rightarrow Op2$ and $Op2 \rightarrow Op3$, then $Op1 \rightarrow Op3$.

Slide 14: Causal Consistency Execution Trace Analysis

Trace 1: Valid Causally Consistent Trace

- P_1: $W(x)a$
- P_2: $R(x)a \rightarrow W(x)b$

  (Causally linked: $W(x)a \rightarrow W(x)b$)
- P_3: $R(x)a \rightarrow R(x)b$ | P_4: $R(x)a \rightarrow R(x)b$

- **Analysis**: VALID!

All processes observe causally linked 'a' before 'b'.

Trace 2: Valid Causal Execution with Concurrent Writes

- P_1: $W(x)a$
- P_2: $W(x)b$

(Concurrent: P_2 did NOT read 'a' before writing 'b')

- P_3: $R(x)a \rightarrow R(x)b$ | P_4: $R(x)b \rightarrow R(x)a$

- **Analysis**: VALID under Causal!

Because $W(a)$ || $W(b)$, P_3 and P_4 may see them in different relative orders.

Slide 15: Implementing Causal Tracking via Vector Timestamps

Capturing Causality in Data Stores

• **Dependency Graph**: Causal consistency requires tracking which processes have observed which write operations.

• **Vector Clock Implementation**: Each write operation carries a vector timestamp representing its causal history.

• **Message Delay Buffer**: When a replica receives a write, it buffers execution until all causally preceding writes are applied.

Vector Comparison Rule for Writes

• **Precedence Condition**: Write W_2 with vector $V(W_2)$ depends on W_1 with $V(W_1)\ iff\ V(W_1) < V(W_2)$.

• **Concurrent Condition**: If neither $V(W_1) \leq V(W_2)$ nor $V(W_2) \leq V(W_1)$, writes are concurrent and can be applied in arbitrary order.

• **Scalability Limit**: Vector size scales linearly with the number of writing processes (N).

Slide 16: Data-Centric Consistency Hierarchy & Trade-off Matrix

Consistency Model	Global Order Guarantee	Concurrent Writes	Implementation Overhead
Strict Consistency	Absolute physical time order	Strictly prohibited	Impossible in asynchronous WANs
Sequential Consistency	Identical global interleave for ALL nodes	Globally ordered identically everywhere	High (Requires total order multicast / locks)
Causal Consistency	Identical order ONLY for causally linked writes	Arbitrary order on different nodes	Moderate (Requires vector timestamp tracking)
FIFO (PRAM) Consistency	Identical order ONLY for writes from SAME process	Arbitrary interleave between processes	Low (Pipelined FIFO channels)

Content

- Module 1: Foundations of Consistency & Continuous Consistency (Slides 01-08).
- Module 2: Strict, Sequential & Causal Consistency Models (Slides 09-16).
- Module 3: FIFO, Weak, Release & Entry Consistency Models (Slides 17-24).
- Module 4: Client-Centric Consistency Models & Session Vectors (Slides 25-32).
- Module 5: Real-World Systems, Review Problems & Practical Lab (Slides 33-40).

Slide 17: FIFO (Pipelined RAM / PRAM) Consistency Mechanics

Formal Definition of FIFO Consistency

• Process-Level Order: Writes done by a single process are seen by all other processes in the order in which they were issued.

• Inter-Process Arbitrary Order: Writes issued by DIFFERENT processes may be observed in different relative orders by different processes.

• Hardware Analogy: Models pipelined RAM memory buffers where memory writes from a single CPU arrive in order.

Implementation Advantages

• Simple Protocol: Implemented using independent FIFO point-to-point network queues between servers.

• No Global Coordination: Process P_1 never stalls waiting for Process P_2's writes to be acknowledged.

• Ultra-High Throughput: Maximum write throughput with minimal network latency.

Slide 18: FIFO Consistency Trace Analysis & The Banking Anomaly

Valid FIFO Execution Trace

- P_1: $W(x)a$
- P_2: $R(x)a \; -> \; W(x)b$
- P_3: $R(x)b \rightarrow R(x)a$ | P_4: $R(x)a \rightarrow R(x)b$

- **Analysis**: VALID under FIFO!
P_3 and P_4 observe writes from different processes in different relative orders.

The Banking Balance Anomaly

- *Initial Balance*: \$100. P_1 deposits \$20 (W(bal)120). P_2 adds 10% interest (W(bal)110).
- *Replica 1 Execution*: Deposit then Interest → Final Balance = \$132.
- *Replica 2 Execution*: Interest then Deposit → Final Balance = \$130.
- **Data Divergence**: FIFO permits permanent data store divergence for financial updates!

Slide 19: Weak Consistency: Synchronization Variables & Memory Barriers

Motivation for Weak Consistency

• Performance Overhead: Enforcing operation-level ordering on every individual read/write is unnecessarily slow.

• Programmer Intent: Applications perform series of local reads/writes, needing global consistency only at critical boundaries.

• Synchronization Variable (S): Introduces explicit synchronization operations (memory fences/barriers) into the data store.

Properties of Weak Consistency

• 1. Access to synchronization variables S is sequentially consistent across all nodes.

• 2. No access to S is permitted until all previous local writes have been flushed everywhere.

• 3. No access to data items is permitted until all previous accesses to S have fully completed.

Slide 20: Weak Consistency Valid vs. Invalid Trace Analysis

Valid Weak Consistency Sequence

- P_1: $W(x)a \rightarrow W(x)b \rightarrow S$
- P_2: $S \rightarrow R(x)b$

- **Analysis**: VALID!

Synchronization variable S flushes P_1's updates to P_2 before P_2 reads x. P_2 reads correct value 'b'.

Invalid Weak Consistency Sequence

- P_1: $W(x)a \rightarrow W(x)b \rightarrow S$
- P_2: $R(x)a \rightarrow S$

- **Analysis**: INVALID!

P_2 attempts to read data item x BEFORE executing synchronization variable S, resulting in an undefined/stale read.

Slide 21: Release Consistency: Explicit Acquire/Release Semantics

Fine-Grained Critical Section Synchronization

• Concept: Replaces generic sync variables with explicit lock primitives: `Acquire(L)` and `Release(L)`.

• `Acquire(L)`: Requests lock L. Ensures local process state is updated with all global writes associated with lock L.

• `Release(L)`: Releases lock L. Forces all local writes executed within the critical section to be propagated to all other replicas.

Eager vs. Lazy Release Protocols

• Eager Release Consistency: Pushes all local writes to all replicas immediately when `Release(L)` is executed.

• Lazy Release Consistency: Delays pushing updates until another process actually executes `Acquire(L)` on the same lock.

• Efficiency Gain: Lazy release avoids network broadcasts if no other process attempts to acquire lock L.

Slide 22: Eager vs. Lazy Release Consistency Protocols

Eager Release Execution Flow

- P_1: $Acquire(L) \rightarrow W(x)a \rightarrow Release(L)$— $[Broadcast\ Updates]$ – $\rightarrow$ $All\ Replicas$
- P_2: $Acquire(L) \rightarrow R(x)a$
- **Network Cost**: High broadcast frequency on every lock release.

Lazy Release Execution Flow

- P_1: $Acquire(L) \rightarrow W(x)a \rightarrow Release(L)$ (No network activity!)
- P_2: $Acquire(L)$— — $[Fetch\ Updates\ from\ P_1]$ – $\rightarrow$ $P_1\ sends\ W(x)a \rightarrow$ $P_2\ executes\ R(x)a$
- **Network Cost**: Targeted point-to-point sync only between previous releaser and new acquirer.

Slide 23: Entry Consistency: Lock-Bound Data Objects

Binding Synchronization Locks to Data Items

• **Concept**: Associates a specific synchronization lock L explicitly with an individual data item or object x.

• **Targeted Sync**: Acquiring Lock L_x synchronizes ONLY data item x, rather than flushing the entire global database memory.

• **Parallel Execution**: Process P_1 acquires L_x to modify 'x' while Process P_2 acquires L_y to modify 'y' simultaneously.

Exclusive vs. Shared Lock Modes

• **Exclusive Acquire**: Required for write access; no other process can hold lock L_x concurrently.

• **Shared Acquire**: Permitted for concurrent read-only access across multiple reader processes.

• **Programming Overhead**: Developers must explicitly declare data-to-lock bindings.

Slide 24: Data-Centric Consistency Models Taxonomy Summary

Model Family	Consistency Model	Synchronization Mechanism	Primary Advantage
Order-Based	Sequential Consistency	Global operation interleave	Simple mental model for developers
Order-Based	Causal Consistency	Vector timestamp tracking	High concurrency for independent writes
Order-Based	FIFO (PRAM) Consistency	Per-process FIFO queues	Maximum throughput; zero cross-node stalls
Group-Based	Weak Consistency	Generic Sync Barrier S	Batched memory flushes
Group-Based	Release Consistency	`Acquire(L)` / `Release(L)`	Lazy point-to-point lock synchronization
Group-Based	Entry Consistency	Lock-bound data objects L_x	Independent parallel updates on separate items

Content

- Module 1: Foundations of Consistency & Continuous Consistency (Slides 01-08).
- Module 2: Strict, Sequential & Causal Consistency Models (Slides 09-16).
- Module 3: FIFO, Weak, Release & Entry Consistency Models (Slides 17-24).
- Module 4: Client-Centric Consistency Models & Session Vectors (Slides 25-32).
- Module 5: Real-World Systems, Review Problems & Practical Lab (Slides 33-40).

Slide 25: Client-Centric Consistency: Eventual Consistency Foundations

The Eventual Consistency Model

• **Definition**: If no new updates are made to a data item, EVENTUALLY all replicas will converge and return identical values.

• **Target Systems**: High read volume, rare write updates, tolerance for temporary inconsistency (e.g., DNS, Web caches, Amazon Dynamo).

• **Cheap Implementation**: Eliminates expensive wide-area synchronous write locking.

The Client-Centric Perspective

• **Problem**: Mobile/disconnected users connect to different server replicas over time, observing inconsistent time jumps.

• **Client-Centric Goal**: Guarantee consistent data views for a SINGLE CLIENT process traversing different replicas over time.

• **Isolation**: Ignores peer process concurrency; focuses entirely on single-client session guarantees.

Slide 26: The Mobile / Portable Client Execution Context

Mobile Client Traversal Scenario

• Step 1: Mobile user connects to Replica Server S1 at home and updates a shopping cart (Write W1).

• Step 2: User disconnects, travels to work, and transparently connects to Replica Server S_2.

• Step 3: If Server S_2 has not received W_1 via background propagation, user sees an empty cart! (Anomaly).

Enforcing Session Guarantees

• Session Context: Client library maintains a session context tracking read/write history.

• Server Validation: Replica S_2 must validate client session history before fulfilling queries.

• 4 Guarantees:

- Monotonic Reads,
- Monotonic Writes,
- Read-Your-Writes,
- Writes-Follow-Reads.

Slide 27: Monotonic Reads Guarantee & Email Refresh Anomaly

Formal Definition of Monotonic Reads

• **Guarantee**: If a process reads the value of data item x, any successive read on x by that process will always return that same value or a more recent value.

• **Rule**: A client will NEVER observe an older version of data than it has already seen in a previous read.

Email Inbox Refresh Anomaly

1. User reads email inbox on Replica S_1 (sees 5 new emails; Version 2).

2. User refreshes page; load balancer routes request to slow Replica S_2.

3. Replica S_2 holds Version 1 (3 emails). Inbox drops 2 emails! (Monotonic Reads violated).

• **Fix**: Replica S_2 delays serving the read until its local state applies Write Set $WS(x_1)$.

Slide 28: Monotonic Writes Guarantee & Software Deployment Order

Formal Definition of Monotonic Writes

• **Guarantee**: A write operation by a process on data item x is completed on any replica before any successive write on x by the same process can be performed.

• **Rule**: Successive writes by the same client are serialized and applied in the exact order issued, regardless of target replica.

Software Library Deployment Anomaly

1. Developer updates core C library (Write W_1 on Replica S_1).

2. Developer updates application binary depending on library (Write W_2 on Replica S_2).

3. Anomaly: If Replica S_3 receives W_2 before receiving W_1, the application crashes due to missing dependencies!

• **Fix**: Replica S_2 holds W_2 until W_1 is fetched and applied first.

Slide 29: Read-Your-Writes Guarantee & Web Profile Update Anomaly

Formal Definition of Read-Your-Writes

• **Guarantee**: The effect of a write operation by a process on data item x will always be seen by a successive read operation on x by that same process.

• **Rule**: A client will NEVER observe a stale data version that omits its own previous updates.

Web Profile Password Update Anomaly

1. User changes account password on Replica S_1 (Write W_1).

2. User attempts to log in on Replica S_2 (Read R_1).

3. **Anomaly**: Replica S_2 rejects login because password update W_1 has not arrived yet! (Read-Your-Writes violated).

• **Fix**: Sticky session routing or write-set header verification.

Slide 30: Writes-Follow-Reads (Causal Client) Guarantee

Formal Definition of Writes-Follow-Reads

• **Guarantee**: A successive write operation by a process on data item x following a previous read on x by the same process is guaranteed to take place on the same or a more recent version of x than was read.

• **Rule**: A client's write reacting to read value v_1 is applied ONLY to replicas that have already processed the write that produced v_1.

Forum Reply Dependency Anomaly

1. User reads Post P_1 on Replica S_1 (produced by Write W_1).

2. User posts Reply R_1 on Replica S_2 (Write W_2 reacting to P_1).

3. Anomaly: If Replica S_3 receives W_2 before W_1, Reply R_1 appears as an orphan thread pointing to a missing parent post!

• **Fix**: Enforce causal write-set dependencies before applying W_2.

Slide 31: Implementing Client-Centric Models with Session Vector Timestamps

Session Vector Architecture

• **Client Session Context**: Client library maintains two vector timestamps: V_{read} and V_{write}.

• V_{read}: Vector clock representing the latest write versions observed by client reads.

• V_{write}: Vector clock representing the latest writes issued by the client.

Server Request Validation Protocol

• *Step 1*: Client attaches V_{read} and V_{write} to every RPC query sent to Replica S_i.

• *Step 2*: Replica S_i compares its local clock V_{server} against incoming client vectors.

• *Step 3*: If $V_{server} \geq V_{read}$ (or V_{write}), S_i executes query immediately.

• *Step 4*: If $V_{server} < V_{read}$, S_i stalls query and fetches missing write sets from peer replicas.

Slide 32: Client-Centric Consistency Guarantees Summary Matrix

Consistency Model	Core Guarantee to Single Client	User-Visible Anomaly Prevented	Enforcement Requirement
Monotonic Reads	Read values never move backward in time	Inbox items disappearing on page refresh	Server state $\geq V_{read}$ before read
Monotonic Writes	Successive writes serialized in issue order	App binary updating before dependent C library	Server holds W_2 until W_1 applied
Read-Your-Writes	Client always observes its own updates	Password update disappearing on login	Sticky routing or write-set fetch
Writes-Follow-Reads	Writes applied after reads execute on updated state	Forum reply appearing before parent post	Causal write-set dependency check

Content

- Module 1: Foundations of Consistency & Continuous Consistency (Slides 01-08).
- Module 2: Strict, Sequential & Causal Consistency Models (Slides 09-16).
- Module 3: FIFO, Weak, Release & Entry Consistency Models (Slides 17-24).
- Module 4: Client-Centric Consistency Models & Session Vectors (Slides 25-32).
- Module 5: Real-World Systems, Review Problems & Practical Lab (Slides 33-40).

Slide 33: Real-World Case Study: Amazon Dynamo Version Vectors

Amazon Dynamo Architecture

• **System Type**: Highly available, eventually consistent distributed key-value store powering e-commerce shopping carts.

• **SLA Requirement**: Always writable (100% write availability, even during network partitions).

• **Conflict Resolution**: Uses Vector Clocks to track concurrent write versions and detect branching histories.

Shopping Cart Reconciliation

• **Branching Carts**: Under network split, two replicas independently modify a cart (producing Vector Clock V_1 and V_2).

• **Client-Side Merge**: On next read, Dynamo returns both conflicting cart versions to the client application.

• **Reconciliation Rule**: Application logic merges both carts (union of items) and writes back reconciled version V_3.

Slide 34: Real-World Case Study: Bayou Disconnected Mobile Storage

Bayou Mobile Database System

• **System Type**: Replicated database designed for weakly connected mobile devices (laptops/PDAs over ad-hoc Wi-Fi).

• **Tentative Updates**: Writes executed locally on mobile nodes are tagged as 'Tentative' until committed.

• **Primary Commit Server**: A single Primary node orders all tentative writes into a permanent canonical log.

Dependency Checks & Merge Procedures

• **Application-Specific Merging**: Each write includes a Dependency Check and an automatic Merge Procedure (executes SQL/script).

• **Conflict Handling**: If dependency check fails (e.g., meeting room double-booked), merge procedure auto-reschedules meeting.

• **Eventual Convergence**: All nodes eventually receive the primary's committed log and converge identically.

Slide 35: Numerical Review Problems & Vector Comparisons

Problem 1: Vector Timestamp Causality Evaluation

• Given 3 process vector timestamps:

$$V(Op_1) = (2, 0, 1),$$
$$V(Op_2) = (3, 1, 1),$$
$$V(Op_3) = (2, 2, 0).$$

• **Question A**: Are Op_1 and Op_2 causally related? Explain. → YES!

$$V(Op_1) < V(Op_2)\ (2 <= 3, 0 <= 1, 1 <= 1)$$

Op_1 causally preceded Op_2.

• **Question B**: Are Op_1 and Op_3 causally related? Explain. → NO!

$Op_1[2] = 1 > Op_3[2] = 0,$
but $Op_1[1] = 0 < Op_3[1] = 2.$

Concurrent $(Op_1 \parallel Op_3)$!

Problem 2: Client Session Vector Validation

• Client $V_{read} = (4, 2)$.

Replica S_1 local clock = (4, 1).

Replica S_2 local clock = (5, 3).

• **Question**: Which replica can serve the client's read under Monotonic Reads?

→ Replica S_2!

$S_2(5,3) \geq V_{read}(4,2).$

Replica S_1 $(4,1)$ must stall.

Slide 36: Conceptual & Analytical Review Questions

Self-Assessment Question 1

• Explain why Sequential Consistency is impossible to guarantee under eventual consistency client-side caching.

• **Model Answer**: Eventual consistency permits temporary replica divergence. Sequential consistency requires an identical global interleave for ALL nodes.

Self-Assessment Question 2

• Contrast Monotonic Reads with Read-Your-Writes consistency.

• **Model Answer**: Monotonic Reads prevents time moving backward across successive reads. Read-Your-Writes ensures a client's reads reflect its OWN previous writes.

Slide 37: Practical Lab Specifications: Client-Centric Session Proxy

Lab Assignment Overview

• Assignment Title: Client-Centric Consistency Session Proxy Simulator.

• Language: Python, Java, or Go.

• System Architecture: Implement a client proxy sitting between client applications and 3 replicated server stores (S1, S2, S3).

Functional Requirements

• 1. Proxy tracks client session vectors (V_{read}, V_{write}).

.2• Implement Monotonic Reads and Read-Your-Writes enforcement logic.

• 3. Simulate network delay/divergence across replicas S1, S2, S3.

• 4. Verify proxy stalls/fetches write sets when client routes to stale replica.

Slide 38: Lab Expected Execution Trace & Evaluation Rubric

Sample Console Execution Trace

```
[CLIENT] Write W1 (Profile Name = 'Alice') -> Sent to
Server S1

[S1] Write ACK | Client V_write updated to (1, 0, 0)

[SYSTEM] Routing Client Read R1 to Server S2... S2 Clock =
(0, 0, 0)

[PROXY] Validation Failed! S2 Clock (0,0,0) < Client
V_write (1,0,0)

[PROXY] Fetching missing Write Set W1 from S1 -> Applying
to S2...

[S2] State Updated to (1, 0, 0) | Read R1 Executed ->
Returns 'Alice'

[CLIENT] Read-Your-Writes Guarantee Preserved!
```

Grading Rubric

- Correct session vector tracking & validation logic (40%)
- Monotonic Reads & Read-Your-Writes enforcement (30%)
- Stale replica write-set fetching & recovery (20%)
- Code documentation & design report (10%)

Slide 39: Consistency Model Hierarchy Summary

Data-Centric Hierarchy (Strongest to Weakest)

1. Strict Consistency (Physical time ordering - theoretical limit).

2. Sequential Consistency (Global operation interleave consensus).

3. Causal Consistency (Causally linked order; concurrent writes split).

4. FIFO (PRAM) Consistency (Process-level FIFO order).

5. Weak / Release / Entry Consistency (Explicit synchronization barriers).

Client-Centric Hierarchy (Session-Based)

1. Monotonic Reads (No backward time reads).

2. Monotonic Writes (Serialized client write streams).

3. Read-Your-Writes (Immediate observation of client updates).

4. Writes-Follow-Reads (Causal client response dependencies).

Slide 40: Lecture 07 Summary & Transition to Lecture 08

Lecture 07 Summary Takeaways

• Consistency Contracts: Consistency models define formal contracts balancing application predictability against network latency.

• Data-Centric Models: Focus on system-wide multi-process operation ordering.

• Client-Centric Models: Focus on single-client session guarantees across traversing mobile clients.

• Session Vectors: Compact vector timestamps elegant enforce client-centric guarantees.

Transition to Lecture 08

• Coming Up in Lecture 08: Consistency Protocols & Implementation Algorithms.

• Key Topics: Primary-Backup protocols, Active Replication, Gifford's Weighted Quorums, Cache-Coherence, and Gossip Protocols.

Recommended Supplementary Resources

1. Distributed Mutual Exclusion Algorithms

• **Ricart, G., & Agrawala, A. K. (1981).** *"An Optimal Algorithm for Mutual Exclusion in Computer Networks". Communications of the ACM*, 24(1), 9–17. (The landmark paper introducing the permission-based, non-token $(2(N-1))$ message algorithm using Lamport logical timestamps to enforce total ordering.)

• **Lin, S.-D., Lian, Q., Chen, M., & Zhang, Z. (2004).** *"A Practical Distributed Mutual Exclusion Protocol in Dynamic Peer-to-Peer Systems". Proceedings of IPTPS*, **11–21.** (Details the decentralized majority-voting algorithm $((m > n/2)$ coordinators) over Distributed Hash Tables (DHTs) covered in Section 6.3.3.)

• **Saxena, P., & Rai, J. (2003).** *"A Survey of Permission-based Distributed Mutual Exclusion Algorithms". Computer Standards & Interfaces*, 25(2), 159–181. (A thorough comparative survey analyzing the performance, message complexity, and failure modes of token-based vs. permission-based mutual exclusion protocols.)

2. Distributed Leader Election Protocols

• **Garcia-Molina, H. (1982).** *"Elections in a Distributed Computing System". IEEE Transactions on Computers, C-31(1), 48–59.* (The original paper introducing the Bully Algorithm $((O(N^2))$ worst-case message complexity).)

• **Vasudevan, S., Kurose, J. F., & Towsley, D. F. (2004).** *"Design and Analysis of a Leader Election Algorithm for Mobile Ad Hoc Networks". Proceedings of IEEE ICNP*, 350–360. (Explains how elections operate in wireless and mobile sensor environments by constructing Directed Acyclic Graph (DAG) spanning trees to elect the node with the best resource capacity.)

• **Ongaro, D., & Ousterhout, J. (2014).** *"In Search of an Understandable Consensus Algorithm" (Raft Protocol). Proceedings of USENIX ATC*. (Modern industrial reading on how leader elections, randomized heartbeat timers, and log replicated state machines function in modern cloud clusters.)